Voltage Fault Injection Against On-Device Language Models

Anthony Etim
Yale University
New Haven, Connecticut, USA
anthony.etim@yale.edu

Jakub Szefer
Northwestern University
Evanston, Illinois, USA
jakub.szefer@northwestern.edu

Abstract

Small transformer language models are today able to run on microcontrollers, where an attacker with physical access can disturb the supply voltage during text generation. Fault injection studies of neural networks in the past have mostly targeted classifiers and modeled the attacker as flipping a chosen bit of a chosen parameter. A voltage fault injection is instead aimed at corrupting whatever the processor is executing and targets the execution of the model. This work characterizes voltage fault injection against a 4,192-parameter autoregressive transformer with a key–value (KV) cache on an Arm Cortex-M4. The language model is augmented with triggers that mark six stages of the forward pass, and the work searches each stage for timing offsets that cause faults. Every stage is found to have an offset at which 43.0% to 99.7% of voltage fault injection glitches produce an anomaly without a crash. The vulnerable offsets depend on the input: on one prompt an offset in the feed-forward network, or multilayer perceptron (MLP), faulted on 299 of 300 attempts, yet on a second prompt it produced no anomalies while other MLP offsets became vulnerable. A single glitch can also leave the current token unchanged and change a later one, consistent with corrupted state carried between tokens. Finally, this work evaluates temporal redundancy that recomputes each token and compares all logits, assuming the recomputation is not itself faulted. No glitch in 6,100 attempts changed the emitted token without an alarm, but at the MLP offset the check alarmed on 299 of 300 attempts that left the output unchanged. These results suggest that defenses for on-device language models should protect the computation in every stage and the state carried between tokens, not only the stored weights.

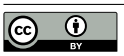
CCS Concepts

• **Security and privacy → Hardware attacks and countermeasures**.

Keywords

Fault injection attacks, Embedded machine learning, LLMs

ACM Reference Format:
Anthony Etim and Jakub Szefer. 2026. Voltage Fault Injection Against On-Device Language Models. In *International Workshop on Hardware and Architectural Support for Security and Privacy 2026 (HASP '26), October 31–November 01, 2026, Athens, Greece.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3857307.3857310



This work is licensed under a Creative Commons Attribution 4.0 International License.
HASP '26, Athens, Greece
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2964-5/2026/10
https://doi.org/10.1145/3857307.3857310

1 Introduction

Transformer language models are being made small enough to run on edge devices: quantization makes local inference practical on boards such as the Raspberry Pi [16], and deliberately small implementations fit on microcontrollers. An attacker who has physical access to the device can do more than choose its input. For example, the attacker can disturb the supply voltage while the device generates text. Text generation is a long computation that proceeds step by step. Each step writes state, such as the key–value (KV) cache, and later steps read that state.

Most fault studies of neural networks target classifiers and model the fault as a flip of a chosen bit in a chosen parameter [12, 26, 31], and recent work applies this model to large language models [5, 9, 30]. This bit-flip model follows Rowhammer [18], which corrupts specific bits stored in memory. A voltage fault injection, instead, does not select a bit: the attacker picks a pulse shape and a time after a trigger, and the instructions and data in flight at that time determine what is corrupted. Physical fault injection has been demonstrated against embedded neural networks and TinyML classifiers with laser, electromagnetic, voltage, and clock faults [3, 6–8, 11, 14], but not against the stages of an autoregressive transformer or the state it carries between tokens.

This paper characterizes where in a transformer's forward pass a voltage glitch causes errors and what those errors look like at the output; it does not build an end-to-end attack that forces a chosen output. We port the inference code of a one-layer, 4,192-parameter, character-level transformer [17] to an Arm Cortex-M4, add a firmware trigger that marks any one of six stages, and measure how long each stage runs. We scan each stage with ten glitch attempts per timing offset and re-test each offset that faulted with 300 attempts. The firmware also implements a candidate defense, a temporal-redundancy *check*: it runs each token's forward pass twice and raises an *alarm* when the two logit vectors disagree. This recomputation gives us a second fault indicator alongside the emitted token and the logits.

At the selected offset in each stage, 43.0% to 99.7% of attempts produce a non-crash anomaly: the device keeps running, but the check raises an alarm, the token changes, or the logits read back are wrong. The vulnerable offsets depend on the input. The scans use short character strings as prompts; the offset that faults almost every time under the prompt an stops faulting under the prompt be, where other feed-forward (MLP) offsets fault instead. One glitch can also leave the current token unchanged and change a later one. Assuming the check's own recomputation is fault-free, no glitch in 6,100 attempts changed the emitted token without an alarm, but at the MLP offset the check alarmed on 299 of 300 attempts that left the token unchanged. The model is small and all data come from one board; Section 6 discusses the limitations of the work. Our contributions are:

- A stage-level characterization of voltage glitching on a Cortex-M4 running a small transformer with a KV cache, using per-stage triggers and execution windows, a timing search followed by 300-attempt re-tests, and outcome categories that keep alarms, token changes, wrong logits, and crashes separate (Sections 4, 5.1, and 5.2).
- Evidence that the vulnerable offsets depend on the prompt, so a timing profile found with one prompt may fail on another (Section 5.2).
- An evaluation of a temporal-redundancy check that separates its alarms from changes to the emitted token (Section 5.3).
- A demonstration that one glitch can leave the current token unchanged and change a later one, consistent with corrupted state carried between tokens (Section 5.4).

2 Background and Related Work

2.1 Transformer Inference and Persistent State

A decoder-only transformer embeds the current token and its position, computes query, key, and value projections, and combines them with causal attention, in which the current position reads the keys and values of itself and all earlier positions but not later ones; it then applies a feed-forward (MLP) block, and a final language-model (LM) head projects the result to vocabulary logits [27]. Implementations cache the keys and values of earlier positions to avoid recomputing them, and serving systems manage this KV cache as a memory object in its own right [20]. A transient fault can therefore affect both the current token and the state that later tokens read. The stages of the forward pass also use the hardware differently: the embedding lookup and cache writes are mostly memory accesses, the projections are multiply–accumulate loops, softmax combines reductions with exponentials, and the MLP is the longest arithmetic stage on our target. We therefore measure each stage separately.

2.2 Physical Fault Injection and Temporal Redundancy

Voltage glitching briefly pulls down a device's supply voltage, which can skip an instruction, corrupt a register or memory value, disrupt control flow, or crash the device [4, 8]. A crowbar glitch, which we use, does this by briefly shorting the supply rail to ground [24]. We use the ChipWhisperer platform [25], which sets a glitch with four parameters: *width* and *phase offset* within a clock cycle, a *repeat* count (how many consecutive cycles receive a pulse), and *ext_offset*, the number of cycles from an external trigger to the glitch. The first three shape the disturbance, and *ext_offset* times it. Below, *phase offset* always means the position within a cycle, and *offset* alone means *ext_offset*. Without an instruction trace, an offset tells us when a glitch occurred but not which instruction it corrupted, and repeated attempts at one setting can have different outcomes, so we report outcome distributions.

The defense we evaluate is temporal redundancy: save the state before a token step, run the forward pass, restore the saved state, run the pass again, and compare the logits. Re-execution is a standard concurrent error-detection technique [10]; we evaluate it assuming that the second pass and the comparison are not themselves faulted.

2.3 Faults in Neural Networks and Language Models

Flipping chosen parameter bits can severely degrade a neural network [12, 26], and Rowhammer can flip such bits on real systems: DeepHammer in dynamic random-access memory (DRAM) [31], and GPUHammer in the GDDR6 memory of an NVIDIA A6000 graphics processing unit (GPU), where the flips reduce model accuracy by up to 80% [21]. Rowhammer needs only code running on the same machine, and a flipped weight stays wrong until it is reloaded. A voltage glitch needs physical access and lasts only for the pulse, but it can corrupt anything the processor computes or writes in that time, including activations and cache entries.

Physical fault injection has also targeted embedded inference. Breier et al. [3] used laser injection against an embedded neural network, and Hou et al. [14] compared several injection methods on edge-device deep neural network (DNN) inference. Gaine et al. [8] studied single instruction skips, induced by electromagnetic (EM) and laser injection, in a convolutional neural network (CNN) on a Cortex-M4. Recent TinyML work applied voltage and EM injection to several microcontroller models and evaluated randomized self-reduction and majority voting [6]. Hommes et al. [11] used EM injection against a CNN on a Cortex-M4, Kučerák et al. [19] characterized EM faults on a neural accelerator, and Breier et al. [4] studied how number representation affects EM fault sensitivity. These works target classifiers and accelerators, not a model whose state persists from one generated token to the next.

For language models, GenBFA and SBFA flip chosen parameter bits to break the output [5, 9], and BitHydra flips them to raise inference cost [30]. Forget and Rewire hardens transformer linear layers against such attacks [22], and CheckOne detects and mitigates faults in vision transformers [1]. Hossain et al. perturb KV-cache tensors in software [13], and Yamamoto and Matsuura [29] analyze idealized bit flips in shared KV-cache blocks. These attack studies each choose a target, such as a parameter bit or a cache block, and measure the effect of corrupting it. We instead fix one physical fault mechanism and sweep its timing across the whole forward pass, so our results describe where and how often a glitch disturbs inference, not the outcome of an attack on one component.

Physical side channels can recover the architecture, parameters, and inputs of embedded networks [2, 15, 28, 32]. To our knowledge, no prior work reports a stage-level voltage-glitch characterization of an autoregressive transformer language model with a KV cache on a Cortex-M microcontroller.

3 Threat Model

We consider an attacker who physically possesses a device that runs a language model locally. The goal is to make the device emit a token that differs from its fault-free output without setting off an on-device alarm. The attacker can disturb the supply rail, rerun inference on chosen inputs while tuning the glitch, and observe whether the device responds, raises an alarm, or emits a different token. The attacker does not modify the firmware and cannot read or replace the model weights. Confidentiality attacks, model extraction, and invasive probing are out of scope.

Synchronization and crashes. Our glitches are timed from a firmware trigger that marks the start of the targeted stage. A field

attacker cannot add this trigger and would have to time the glitch from an external signal, with jitter. Our rates therefore measure susceptibility under exact timing, not the success rate of a field attack; Section 6 discusses the difference. A crash means that the target stopped responding and had to be reset or reprogrammed. We do not claim permanent damage to the device and did not measure cumulative electrical wear.

Persistent state. A transient glitch can alter state that later tokens read. The KV cache is one such object, but we do not compare memory snapshots or reconstruct the cache, so we cannot attribute a later divergence to the cache as opposed to other memory or control state.

Defense under evaluation. The defense saves the state before a token step, runs a possibly glitched pass, restores the saved state, runs a second pass with triggers disabled, and compares the logits. We do not glitch the second pass, the comparison, or the restore. After the comparison, the firmware restores the glitched pass's state so that we can follow the fault, so the implementation detects faults but does not recover from them.

4 Methodology

We ask four questions. **RQ1**: Can voltage glitching cause non-crash anomalies in the forward pass, and at what crash cost? **RQ2**: How do outcomes differ across stages and inputs? **RQ3**: How do the alarms of the recomputation check relate to changes in the emitted token? **RQ4**: Can one glitch change a later token, as corrupted persistent state would?

4.1 Target Model and Firmware

The target follows Karpathy's `microgpt` [17]: one transformer layer, embedding width n_{embd}=16, four attention heads, MLP width 64, context length 16, and a 27-symbol character vocabulary, trained on personal names. RMSNorm has no parameters and no layer has a bias, so the model has 4,192 learned parameters: 432 in the token embedding (27×16), 256 in the position embedding, 1,024 in the Q/K/V/O projections, 2,048 in the two MLP projections, and 432 in the LM head. It is a toy model, not a representative large language model, but it has the parts of a decoder-only transformer that matter here: token and position embeddings, causal multi-head attention with a KV cache, residual connections, an MLP, and a vocabulary projection. We reimplemented inference in C with the weights compiled into the firmware. On eight test prompts, the firmware predicts the same token as an independent NumPy implementation, with logits that differ by at most 1.4×10^{-6}.

Stage triggers. The firmware holds a trigger line high while it executes either the complete forward pass (WHOLE) or one of six stages: the embedding lookup (EMBED), the Q/K/V projections (QKV), the KV-cache write (KV_WRITE), the attention softmax (SOFTMAX), the MLP (MLP), or the LM head (LM_HEAD). Each experiment marks one region.

Recomputation check. After the first pass, the firmware keeps that pass's output and state, restores the state saved before it, recomputes with triggers disabled, and compares all 27 logits. A difference above $\epsilon = 10^{-5}$ in any logit sets `fault_flag`. The firmware then restores the first pass's state, so generation continues on the faulted path. Because the check covers the full logit vector, it can alarm even when the greedy arg max, and so the emitted token, is unchanged. The second pass adds about 1.63 million cycles per token (221 ms at our clock rate; Table 1), roughly doubling compute time.

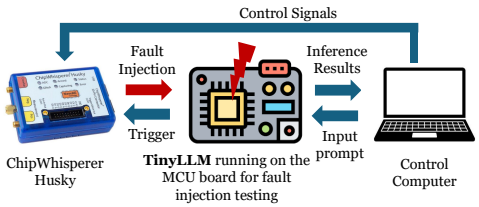


Figure 1: Experiment setup. A ChipWhisperer-Husky supplies the target clock and the crowbar glitch. An Arm Cortex-M4 SAM4S target runs the transformer firmware and holds a trigger line high during the selected region.

4.2 Hardware Setup

Figure 1 shows the setup. A ChipWhisperer-Husky clocks the SAM4S target at 7.3636 MHz and generates the glitch. The firmware is compiled for a nominal $F_{\text{CPU}} = 7.3728$ MHz, which only sets the serial-port (UART) baud-rate divisor; all cycle counts use the 7.3636 MHz clock.

What the glitch does. The Husky's high-power crowbar output (`glitch_only` mode) switches on a power transistor (MOSFET) that briefly shorts the target's supply rail to ground [24]. Width and phase offset are measured in phase steps, 4,592 per clock period. At the setting chosen in Section 4.5, each pulse is 2,100 steps wide (about 46% of a period, or 62 ns) and starts 2,600 steps (about 57%) into the cycle, and a repeat count of 2 applies it in two consecutive cycles. In `ext_single` mode, the Husky waits for the rising edge of the firmware trigger and fires once, *ext_offset* cycles later. The glitch therefore hits whichever instructions of the marked stage execute at that cycle, such as a multiply–accumulate in the MLP or a store into the KV cache. It does not target a bit; which bits go wrong, and how many, depends on what is executing. We see the effect only at the outputs (the emitted token, the read-back logits, the alarm flag, and whether the target responds). We do not trace instructions or registers, so we cannot say which instruction or bit a glitch corrupted.

4.3 Experimental Pipeline and Metrics

For each prompt and token position, a fault-free run gives the reference token and logits. A glitch attempt applies one *glitch parameter setting*, the glitch width, phase offset, external offset, and repeat count that shape the voltage pulse (Section 2). A responsive target returns the emitted token, the logits, and the alarm flag; an unresponsive one is reset or reprogrammed. We first scan each stage's window with ten attempts per sampled offset, which shows whether an offset faults but not how often, and then re-test each offset that faulted with 300 new attempts. Because the scan chose these offsets, their rates describe the most vulnerable offsets we found, not a typical offset in the stage.

Table 1: Execution windows in target clock cycles.

Region	Cycles	Region	Cycles
WHOLE	1,629,016	SOFTMAX	81,572
MLP	781,152	EMBED	16,640
QKV	346,996	KV_WRITE	996
LM_HEAD	196,372		

An attempt can raise an alarm, change the emitted token, change the read-back logits, or crash, and several can happen together. For the outcome tables, we assign each attempt one category, checked in this order: CRASH; ALARM if `fault_flag` is set; SILENT-TOKEN if the emitted token changed without an alarm; READ-BACK-MISMATCH if there was no alarm and the token is unchanged but the logits the host reads back differ from the reference; otherwise NO-EFFECT. A read-back mismatch shows that the host received wrong logits, not whether the error arose in the computation, in stored state, or in transfer.

For N attempts, let n_A, n_S, n_R, and n_C count alarms, silent-token changes, read-back mismatches, and crashes. We call the first three *non-crash anomalies* and report their rate NAR = $(n_A + n_S + n_R)/N$, along with AR = n_A/N, RMR = n_R/N, STR = n_S/N, and CR = n_C/N. The ratio $n_A/(n_A + n_S)$ is not the check's recall for output changes, because n_A includes alarms where the token did not change. Recall is $n_{A\cap T}/(n_{A\cap T} + n_S)$, where T is the set of attempts that changed the emitted token. An alarmed attempt is counted only as an alarm, so we have $n_{A\cap T}$ for only some offsets.

4.4 Device Output and Later-Token Drift

Under a fault, the emitted token and the host's arg max of the read-back logits can disagree. We treat the emitted token as the output, since a user would act on it, and record the host-side arg max with it. The two agree on fault-free runs.

To test propagation, we glitch one token step and let generation continue without glitches for up to twelve more tokens or until the end-of-sequence symbol; we call each such run a *rollout*. In all 596 rollouts, the clean continuation ended after one visible character, so corrupted state can show up only by changing that character or by preventing termination, and a zero drift rate is weak evidence. Among rollouts that survive the glitch, the drift rate (DR) is the fraction whose text differs from the clean run, and the persistent-state drift rate (PDR) is the fraction whose glitched token matches the clean one but whose later text does not. A nonzero PDR is what corrupted inter-token state would produce, but it does not single out the KV cache. We did not record each rollout's starting position or how the context limit was handled, so we treat the rollouts as qualitative evidence.

4.5 Execution Windows and Search

We measure each region's trigger-high time with the cycle counter of the Husky's capture analog-to-digital converter (ADC) (Table 1).

The six stage windows cover 1,423,728 of the 1,629,016 cycles in the whole pass; normalization, residual additions, and attention-score computation take the rest. A coarse whole-pass scan uses an 8,145-cycle stride. The stage scans sample offsets instead of testing every cycle, and we do not list their grids or refinement steps, so a selected offset is one that faulted often in our scan, not necessarily the most sensitive cycle in its stage. We write a test location as region@offset; for example, MLP@531,080 is 531,080 cycles after the MLP trigger rises.

Table 2: Attempts per experiment. The defense evaluation combines the two rows above it, giving the 6,100 attempts in which no silent token change occurred. The other experiments answer different questions and are counted separately.

Experiment	Attempts	Used for
Glitch parameter search	3,000	Choosing width, phase offset, and repeat
Re-tests at eleven offsets	3,300	Outcome rates in Table 3
Fixed-offset input variation	2,800	Whether an offset transfers across prompts
Defense evaluation (both)	6,100	Silent-token rate of the check
Host/device diagnostics	100	Emitted token versus host arg max
Later-token rollouts	46–133/offset	Drift after a single glitch

We chose the glitch parameter setting (the glitch width, phase offset, and repeat count) with a 3,000-attempt Optuna search using the tree-structured Parzen estimator (TPE). It jointly sampled $width \in \{2000, 2010, \ldots, 2290\}$, phase $offset \in [1800, 4000]$, and $repeat \in \{1, 2\}$, gave crashes zero reward, and selected $width = 2100$, phase $offset = 2600$, and $repeat = 2$. Because the three parameters varied together, outcomes grouped by width describe the sampled settings, not the effect of width alone.

4.6 Evaluation Scope and Attempt Counts

The defense evaluation combines the 3,300 re-test attempts at the eleven selected offsets with 2,800 attempts that repeat fixed offsets under other inputs, 6,100 in total. The parameter search, host/device diagnostics, second-prompt scan, and rollouts answer other questions and are counted separately (Table 2). All rates come from one small model on one board, with greedy decoding, at selected offsets, and with an unattacked recomputation pass; Section 6 lists what we did not test.

5 Experimental Results

Unless stated otherwise, all experiments use $width = 2100$, phase $offset = 2600$, $repeat = 2$, greedy decoding, and the recomputation check. Each selected offset is re-tested with $N = 300$ attempts.

5.1 RQ1: Fault and Crash Behavior

Figure 2 summarizes the parameter search, which sampled thirty widths from 2000 to 2290 in steps of 10 while varying phase offset and repeat.

Non-crash anomalies appear at widths 2100 and 2130 (grouped rates of 2.5% and 2.2%) but not at 2110 or 2120. Crash rates are 0 to 17% below 2150, rise to 94% at 2200, and reach 100% from 2220 upward; none of the eight widths between 2130 and 2220 produced a non-crash anomaly. Because phase offset and repeat varied with width, the figure shows which settings worked and where crashes dominate, not a width band that anomalies require.

Crash cost depends strongly on timing. At WHOLE@379,324, the selected setting produces 122 non-crash anomalies and 105 crashes

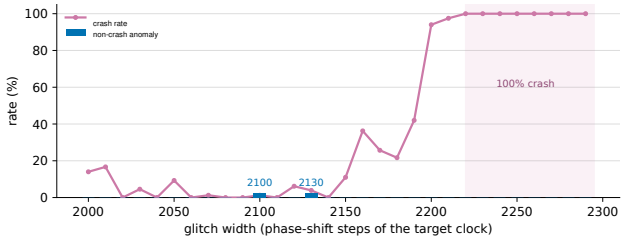


Figure 2: Crash and non-crash anomaly rates by glitch width over the 3,000 attempts of the parameter search. Phase offset and repeat varied along with width, so this is not a controlled sweep of width.

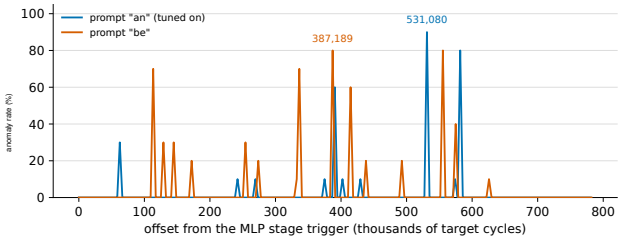


Figure 3: Non-crash anomaly rate at sampled MLP offsets for prompts an and be (ten attempts per offset). Under be, 387,189 is among the highest sampled offsets (8/10). The offset tuned on an, 531,080, is not on the be scan's grid; the fixed-offset campaign records 0/100 there under be.

Table 3: Outcome rates at the eleven selected offsets ($N = 300$ each): non-crash anomaly (NAR), alarm (AR), read-back mismatch with no alarm and an unchanged token (RMR), silent token change (STR), and crash (CR). Each rate is rounded separately; Section 5.5 gives the counts.

Location	NAR	AR	RMR	STR	CR
One selected offset per stage					
MLP@531,080	0.997	0.997	0.000	0.000	0.000
QKV@76,296	0.803	0.803	0.000	0.000	0.167
LM_HEAD@134,397	0.773	0.773	0.000	0.000	0.003
EMBED@8,549	0.747	0.747	0.000	0.000	0.000
SOFTMAX@32,967	0.637	0.637	0.000	0.000	0.270
KV_WRITE@724	0.430	0.430	0.000	0.000	0.450
Alarm-evasion candidates and whole-pass control					
WHOLE@700,470	0.627	0.627	0.000	0.000	0.000
LM_HEAD@22,563	0.453	0.087	0.367	0.000	0.130
WHOLE@379,324	0.407	0.197	0.210	0.000	0.350
LM_HEAD@13,734	0.250	0.000	0.250	0.000	0.580
LM_HEAD@4,905	0.180	0.000	0.180	0.000	0.680

in 300 attempts, while four of the selected offsets crash at most once in 300. A coarse whole-pass scan with an 8,145-cycle stride finds 52 anomalies, 43 of them between 400k and 800k cycles. Later cycles are not safe, however: the stage scans find vulnerable offsets in the MLP and LM_HEAD stages, which run past the 800k point of the whole pass.

5.2 RQ2: Stage and Input Sensitivity

Table 3 gives outcome rates at one selected offset per stage, at four candidate offsets for evading the alarm, and at one whole-pass control; Section 5.5 gives the counts.

At the six stage offsets, NAR ranges from 43.0% at KV_WRITE to 99.7% at MLP. These are rates at offsets the scan selected, not the chance that a glitch anywhere in the stage causes an anomaly. Under the same setting, the outcome mix changes with timing: WHOLE@700,470 has 188 anomalies and no crashes, while LM_HEAD@4,905 has 54 anomalies and 204 crashes. The MLP has both the longest window and the highest selected rate, but window length does not explain the ordering, since EMBED has a higher rate than the longer SOFTMAX stage. Averaged over all sampled offsets in a window, NAR ranges from 5.4% for EMBED down to 0.9% for KV_WRITE, because most offsets produce no anomaly, while mean crash rates stay between 10% and 12.5%. The gap between selected and average rates shows how much an attack depends on finding the right offset. Neither rate identifies a fault mechanism.

The fixed-offset campaign (2,800 attempts) repeats selected offsets with other prompts, faulted tokens, and sequence positions. MLP@531,080 and LM_HEAD@134,397 keep their high rates on the input they were tuned on but produce no anomalies on three other inputs, while EMBED@8,549 stays between 73% and 79% on all four prompts.

We also rescanned the MLP window under a second prompt, be; the first scan used an (Figure 3). Under be, offset 387,189 produces 8/10 anomalies. The original offset, 531,080, is not on this scan's grid. Its nearest sampled neighbor, 531,896, gives 0/10, and the fixed-offset campaign gives 0/100 at 531,080 itself under the same prompt. The second prompt does more than remove the original peak: its scan has 15 offsets with at least one anomaly, against 10 for the first, and a window-average NAR of 2.9% against 1.6%. The MLP stays vulnerable, but at different offsets. This shows input-dependent MLP susceptibility at the offsets we tested. It does not show that every stage behaves this way, or that an offset maps to the same instruction under different inputs.

5.3 RQ3: Detection and Output Impact

No attempt changed the emitted token without an alarm. There were no silent-token changes in the 3,300 re-tests or the 2,800 fixed-offset attempts, 0 of 6,100 in total (Section 4.6). This supports the check as tested, with a fault-free second pass, but it is not a coverage guarantee. Recall for output changes needs the number of token-changing faults that also raised an alarm, which we have only for some offsets, so we report the observed count of zero and its one-sided 95% upper bound of 0.049% per attempt (Section 5.5).

At MLP@531,080, 299 of 300 attempts raise an alarm, yet the emitted token is unchanged in all 300, and a separate rollout experiment at this offset shows no change in the generated text. The rollouts are different attempts, but both results show that a high rate of logit-mismatch alarms does not imply a high rate of changed output. We ran rollouts at seven of the eleven offsets.

Figure 4 shows the outcome mix at all eleven offsets. Four offsets produce read-back mismatches with no alarm and an unchanged token, in 54 to 110 of 300 attempts. In a direct diagnostic at LM_HEAD@4,905, the target survived 24 of 60 attempts. In

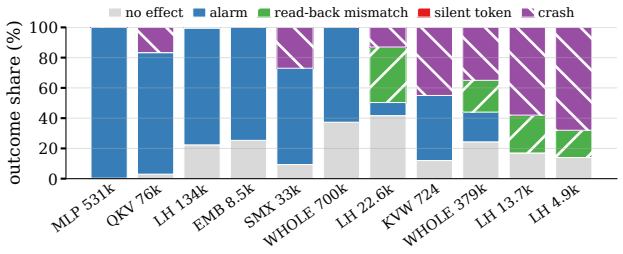


Figure 4: Outcome mix at the eleven selected offsets (3,300 attempts). No attempt falls in the silent-token category.

those 24, the host's arg max of the read-back logits differs from the clean reference 13 times, and the device's emitted token never does. Two immediate reads return identical bytes, which makes a one-off transfer error unlikely, but the cause is open: buffer selection, restore order, serialization, memory aliasing, or a difference between the two arg max implementations could each explain it. At MLP@531,080, host and device agree in all 40 diagnostic attempts. A deployment that exports logits, not just the token, should validate them end to end.

5.4 RQ4: Later-Token Propagation

We glitched one token step at seven offsets and let generation continue (Section 4.4). Excluding crashes leaves 46 to 133 surviving rollouts per offset, so all rates are conditional on survival. At the four stage offsets, 72.1% to 100% of surviving rollouts raise an alarm, yet the generated text matches the clean run at three of them. Of the three LM_HEAD candidates, offsets 4,905 and 13,734 produce no divergence, and offset 22,563 diverges in 7.5% of survivors because the glitched token itself changes.

At QKV@76,296 the PDR is 0.049: the glitched token matches the clean one, and the divergence starts one token later. This fits a fault in the QKV region that leaves the current arg max intact but alters state that the next step reads. All 14 diverging rollouts produced longer text than the clean run, in two ways. The four at QKV@76,296 reproduced the clean first character and then failed to stop, running to the twelve-token cap (`a` versus `aaaaaaaaaaaa`); the ten at LM_HEAD@22,563 substituted the first character and emitted one more (`a` versus `ma`). A glitch can thus suppress termination at one offset and substitute a token at another. Because we did not rule out other persistent state, we do not attribute the QKV effect to the KV cache. No undetected persistent divergence appears in these experiments, but ruling it out would require complete denominators, a record of context handling, attacks on the check path, and other decoding policies.

5.5 Exact Counts and Uncertainty

Table 4 gives the integer counts behind Table 3. Every row satisfies $D = A + R + S$ and $N = D + C + NE$. With $N = 300$, each three-decimal rate corresponds to exactly one count; for example, 0.997 is 299/300 and 0.003 is 1/300.

Table 4: Integer counts for Table 3: non-crash anomalies (D), alarms (A), read-back mismatches (R), silent-token changes (S), crashes (C), and no effect (NE).

Location	D	A	R	S	C	NE
MLP@531,080	299	299	0	0	0	1
QKV@76,296	241	241	0	0	50	9
LM_HEAD@134,397	232	232	0	0	1	67
EMBED@8,549	224	224	0	0	0	76
SOFTMAX@32,967	191	191	0	0	81	28
KV_WRITE@724	129	129	0	0	135	36
WHOLE@700,470	188	188	0	0	0	112
LM_HEAD@22,563	136	26	110	0	39	125
WHOLE@379,324	122	59	63	0	105	73
LM_HEAD@13,734	75	0	75	0	174	51
LM_HEAD@4,905	54	0	54	0	204	42

Table 5: Observed NAR and Wilson 95% interval for $N = 300$.

Location	NAR	Interval	Location	NAR	Interval
MLP@531,080	0.997	[0.981, 0.999]	WHOLE@700,470	0.627	[0.571, 0.679]
QKV@76,296	0.803	[0.755, 0.844]	LM_HEAD@22,563	0.453	[0.398, 0.510]
LM_HEAD@134,397	0.773	[0.723, 0.817]	WHOLE@379,324	0.407	[0.353, 0.463]
EMBED@8,549	0.747	[0.695, 0.793]	LM_HEAD@13,734	0.250	[0.204, 0.302]
SOFTMAX@32,967	0.637	[0.581, 0.689]	LM_HEAD@4,905	0.180	[0.141, 0.227]
KV_WRITE@724	0.430	[0.375, 0.487]			

Table 5 gives Wilson 95% intervals for NAR at the eleven offsets. They describe repeated trials at a fixed offset and do not account for the offset having been chosen by search, for correlation between attempts, or for variation between boards and sessions.

With zero silent-token changes in N attempts, the rule of three gives an approximate one-sided 95% upper bound of $3/N$ per attempt: 0.049% for all 6,100 defense attempts, 0.091% for the 3,300 re-tests, and 0.107% for the 2,800 fixed-offset attempts. These bounds assume independent attempts with a common rate, which is generous because the offsets were chosen by search. None of them bounds the probability that the check misses a fault that does change the token.

6 Discussion

Synchronization without the firmware trigger. The firmware trigger tells the glitcher when a stage starts, and because the Husky also clocks the target, each glitch is aligned to a processor cycle. A field attacker may have neither advantage and would then need an external reference, such as the serial command that starts inference or a feature of the power trace (Fukuda et al. time glitches against a neural network on a microcontroller by matching its power waveform [7]), and a device with its own oscillator adds drift between the reference and the glitch. We did not measure this jitter, but our data bracket its effect. With exact timing at a scan-selected offset, NAR is 43.0% to 99.7%. An attacker who knows when a stage starts but not which cycle in it is vulnerable would see roughly the window average, 0.9% to 5.4%, with 10% to 12.5% crashes. Where a real attack falls depends on how its jitter compares with the width of a vulnerable region, and the isolated peaks in Figure 3 suggest these regions are narrow.

Input dependence. Only the EMBED offset transferred across prompts; the MLP and LM_HEAD offsets did not. Hardening only the cycles found vulnerable for one input would leave a stage exposed

for other inputs, so a defense has to protect the computation and the state it writes, whatever the cycle.

Alarms and responses. The check compares the whole logit vector while greedy decoding uses only the largest logit, so many alarms come from faults that do not change the token. What the system does on an alarm therefore matters as much as detection. Halting on every mismatch turns such faults into denial of service, retrying is unsafe if one disturbance can affect both executions, and voting adds latency and energy. Comparing only the final token reduces false alarms but misses faults that leave the token intact and corrupt state.

Scope of the defense result. The recomputation check is a baseline, not a deployable countermeasure. We assumed that the second pass, the comparison, and the restore cannot be faulted, but an attacker with physical access could glitch them too, and we did not test this. The check also detects faults without recovering from them, and the second pass roughly doubles the compute per token (Section 4).

Limitations and generality. All data come from one SAM4S board, one glitch setting, and a few prompts (two for the MLP timing scans, four for the fixed-offset tests). We did not test other boards, sweep the glitch parameters per stage, evaluate sampled decoding, identify instruction-level fault mechanisms, isolate the KV cache as the cause of later-token drift, vary ϵ, or measure the false-alarm rate without glitches. The one-layer, 4,192-parameter target is far smaller than the language models deployed on phones and edge boards. Model size, tokenization, decoding policy, kernels, and compiler settings all change timing and output impact, so neither our rates nor our stage ordering should be carried over. Sampled decoding, for example, would turn perturbations that preserve the top logit into token changes. Our questions transfer more readily than our numbers: whether each stage has vulnerable offsets, whether they move with the input, and whether one glitch can corrupt state that later tokens read. Answering them for a larger transformer and on more boards is the most direct next step. The method also transfers: separate the timing search from rate estimation, and report alarms, token changes, crashes, and later drift as separate quantities.

7 Conclusion

We characterized voltage glitching against an autoregressive transformer with a key–value cache on an Arm Cortex-M4, across six stages of the forward pass. Using stage triggers and measured execution windows, we found an offset in every stage at which 43.0% to 99.7% of attempts caused a non-crash anomaly. These offsets move with the input prompt, and a single glitch can leave the current token unchanged while changing a later one. With a recomputation check whose own path we assumed fault-free, no glitch in 6,100 attempts changed the emitted token without an alarm, but many alarms came from faults that left the token unchanged. Future work should time glitches without a firmware trigger, attack the check path itself, and design defenses that protect the state carried between tokens.

Acknowledgments

This work was supported in part by National Science Foundation grants 2245344 and 2332406. We would like to acknowledge the free access and use of the Whisp Cloud online service [23] for running the voltage fault injection experiments.

References

[1] Mohammad Hasan Ahmadilivani, Sven-Markus Loorits, and Jaan Raik. 2026. CheckOne: Lightweight Fault Detection and Mitigation for Vision Transformers. *arXiv preprint arXiv:2608.04035* (2026).

[2] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. 2019. CSI NN: Reverse engineering of neural network architectures through electromagnetic side channel. In *28th USENIX Security Symposium*. 515–532.

[3] Jakub Breier, Xiaolu Hou, Dirmanto Jap, Lei Ma, Shivam Bhasin, and Yang Liu. 2018. Practical Fault Attack on Deep Neural Networks. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2204–2206. https://doi.org/10.1145/3243734.3278519

[4] Jakub Breier, Štefan Kučerák, and Xiaolu Hou. 2026. The Weight of a Bit: EMFI Sensitivity Analysis of Embedded Deep Learning Models. *arXiv preprint arXiv:2602.16309* (2026).

[5] Sanjay Das, Swastik Bhattacharya, Souvik Kundu, Shamik Kundu, Anand Menon, Arnab Raha, and Kanad Basu. 2024. GenBFA: An Evolutionary Optimization Approach to Bit-Flip Attacks on LLMs. *arXiv preprint arXiv:2411.13757* (2024).

[6] Anthony Etim, Srilalith Nampally, Aubtin Rasouli, Dustin Mazza, Krishna Chilakapati, Tinghung Chiu, Ferhat Erata, Leyla Nazhandali, Wenjie Xiong, and Jakub Szefer. 2026. Fault Injection Attacks and Countermeasures on TinyML Algorithms. In *2026 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 68–78. https://doi.org/10.1109/HOST68814.2026.11604834

[7] Yuta Fukuda, Kota Yoshida, and Takeshi Fujino. 2022. Fault Injection Attacks Utilizing Waveform Pattern Matching against Neural Networks Processing on Microcontroller. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* E105-A, 3 (2022), 300–310. https://doi.org/10.1587/transfun.2021CIP0015

[8] Clément Gaine, Pierre-Alain Moëllic, Olivier Potin, and Jean-Max Dutertre. 2023. Fault Injection on Embedded Neural Networks: Impact of a Single Instruction Skip. In *26th Euromicro Conference on Digital System Design (DSD)*. 317–324. https://doi.org/10.1109/DSD60849.2023.00052

[9] Jingkai Guo, Chaitali Chakrabarti, and Deliang Fan. 2025. SBFA: Single Sneaky Bit Flip Attack to Break Large Language Models. *arXiv preprint arXiv:2509.21843* (2025).

[10] Xiaofei Guo and Ramesh Karri. 2013. Recomputing with Permuted Operands: A Concurrent Error Detection Approach. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 10 (2013), 1595–1608. https://doi.org/10.1109/TCAD.2013.2263037

[11] Steyn Hommes, Vincent Dankbaar, Tanguy Stekke, Xiaomeng Wang, Lisanne Weidmann, Senna van Hoek, Durba Chatterjee, Lejla Batina, and Zhuoran Liu. 2026. Triggering Stealthy Feature Map Backdoors via Physical Fault Injection in Embedded Neural Networks. *arXiv preprint arXiv:2607.09473* (2026).

[12] Sanghyun Hong, Pietro Frigo, Yiğitcan Kaya, Cristiano Giuffrida, and Tudor Dumitraş. 2019. Terminal Brain Damage: Exposing the Graceless Degradation in Deep Neural Networks Under Hardware Fault Attacks. In *28th USENIX Security Symposium*. 497–514.

[13] Elias Hossain, Swayamjit Saha, Somshubhra Roy, and Ravi Prasad. 2025. Can Transformer Memory Be Corrupted? Investigating Cache-Side Vulnerabilities in Large Language Models. *arXiv preprint arXiv:2510.17098* (2025).

[14] Xiaolu Hou, Jakub Breier, Dirmanto Jap, Lei Ma, Shivam Bhasin, and Yang Liu. 2021. Physical Security of Deep Learning on Edge Devices: Comprehensive Evaluation of Fault Injection Attack Vectors. *Microelectronics Reliability* 120, Article 114116 (2021). https://doi.org/10.1016/j.microrel.2021.114116

[15] Weizhe Hua, Zhiru Zhang, and G. Edward Suh. 2018. Reverse engineering convolutional neural networks through side-channel information leaks. In *55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. 4:1–4:6. https://doi.org/10.1145/3195970.3196105

[16] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Kåsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. 2025. Sustainable LLM Inference for Edge AI: Evaluating Quantized LLMs for Energy Efficiency, Output Accuracy, and Inference Latency. *ACM Transactions on Internet of Things* 6, 4, Article 28 (2025), 35 pages. https://doi.org/10.1145/3767742

[17] Andrej Karpathy. 2026. microgpt: The most atomic way to train and run inference for a GPT in pure, dependency-free Python. Author blog. https://karpathy.github.io/2026/02/12/microgpt/

[18] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. 2014. Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 361–372.

[19] Štefan Kučerák, Jakub Breier, and Xiaolu Hou. 2026. Characterizing the Fault Response of the Intel Neural Compute Stick 2 Under Single-Pulse Electromagnetic Fault Injection. *arXiv preprint arXiv:2605.22437* (2026).

[20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient

Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. 611–626.

[21] Chris S Lin, Joyce Qu, and Gururaj Saileshwar. 2025. {GPUHammer}: Rowhammer attacks on {GPU} memories are practical. In *34th USENIX Security Symposium (USENIX Security 25)*. 5719–5738.

[22] Najmeh Nazari, Hosein Mohammadi Makrani, Chongzhou Fang, Hossein Sayadi, Setareh Rafatirad, Khaled N. Khasawneh, and Houman Homayoun. 2024. Forget and Rewire: Enhancing the Resilience of Transformer-based Models against Bit-Flip Attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 1349–1366.

[23] Northwestern University. 2025. Whisp Cloud: On-demand cloud-based access to side-channel analysis and fault-injection tools. Retrieved October 17, 2026 from https://whisp.cloud

[24] Colin O'Flynn. 2016. Fault injection using crowbars on embedded systems. *Cryptology ePrint Archive* (2016).

[25] Colin O'Flynn and Zhizhang Chen. 2014. ChipWhisperer: An open-source platform for hardware embedded security research. In *International Workshop on Constructive Side-Channel Analysis and Secure Design (COSADE)*. Springer, 243–260. https://doi.org/10.1007/978-3-319-10175-0_17

[26] Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. 2019. Bit-Flip Attack: Crushing Neural Network with Progressive Bit Search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 1211–1220. https://doi.org/10.1109/ICCV.2019.00130

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[28] Lingxiao Wei, Bo Luo, Yu Li, Yannan Liu, and Qiang Xu. 2018. I know what you see: Power side-channel attack on convolutional neural network accelerators. In *34th Annual Computer Security Applications Conference (ACSAC)*. 393–406. https://doi.org/10.1145/3274694.3274696

[29] Yuji Yamamoto and Satoshi Matsuura. 2026. Bit-Flip Vulnerability of Shared KV-Cache Blocks in LLM Serving Systems. *arXiv preprint arXiv:2604.17249* (2026). Version 2.

[30] Xiaobei Yan, Yiming Li, Hao Wang, Han Qiu, and Tianwei Zhang. 2025. BitHydra: Towards Bit-flip Inference Cost Attack against Large Language Models. *arXiv preprint arXiv:2505.16670* (2025). Version 4.

[31] Fan Yao, Adnan Siraj Rakin, and Deliang Fan. 2020. DeepHammer: Depleting the Intelligence of Deep Neural Networks through Targeted Chain of Bit Flips. In *29th USENIX Security Symposium*. 1463–1480.

[32] Honggang Yu, Haocheng Ma, Kaichen Yang, Yiqiang Zhao, and Yier Jin. 2020. DeepEM: Deep neural networks model recovery through EM side-channel information leakage. In *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 209–218.